MLOPS

Data Versioning Visual Guide

Reproducible ML starts with data:
DVC, LakeFS, snapshots, lineage.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/5

Why Version Data

Code versioning alone can't reproduce a model.

Without

'train.csv' changed

under you. Results unreproducible.

With

data@a3f9c1 ->

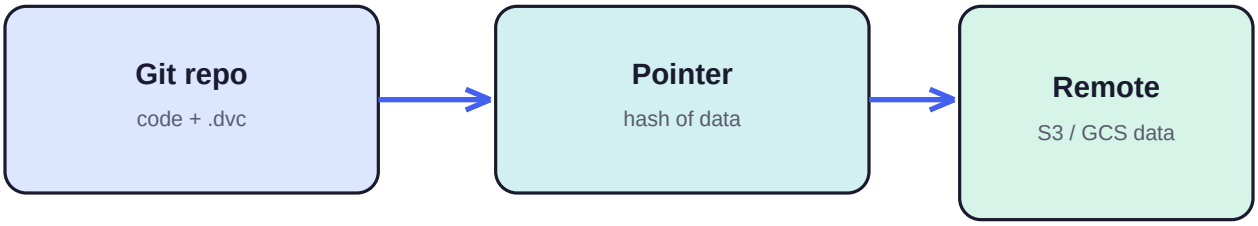
model v2.1. Exact snapshot, every time.

The number-one reproducibility failure: the same code, 'the same data', different results — because the data wasn't the same.

STEP 2/5

The DVC Model

Git for code, pointers for data.



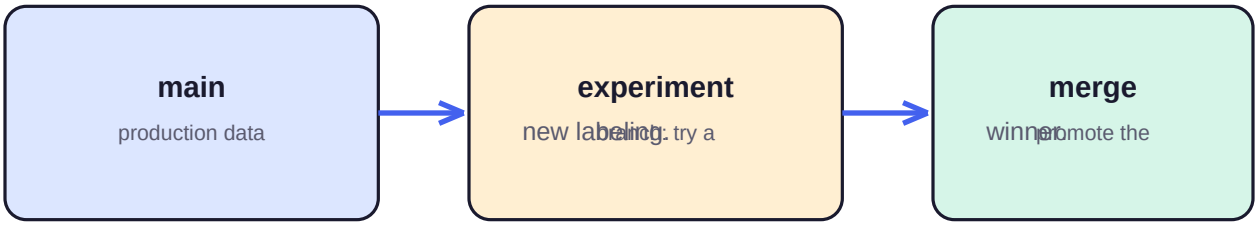
DVC stores tiny pointer files in git; the bytes live in remote storage, fetched by hash on demand.

Checking out a commit checks out the data pointer too — code and data stay in lockstep.

STEP 3/5

LakeFS: Git for Data Lakes

Branches and merges, but for datasets.



Zero-copy branches: experimenting on a data branch doesn't duplicate terabytes — until you commit changes.

STEP 4/5

The Reproducibility Recipe

Four ingredients, all pinned.

Code

git commit hash

Data

snapshot hash / version

Environment

container image + lockfile

Config

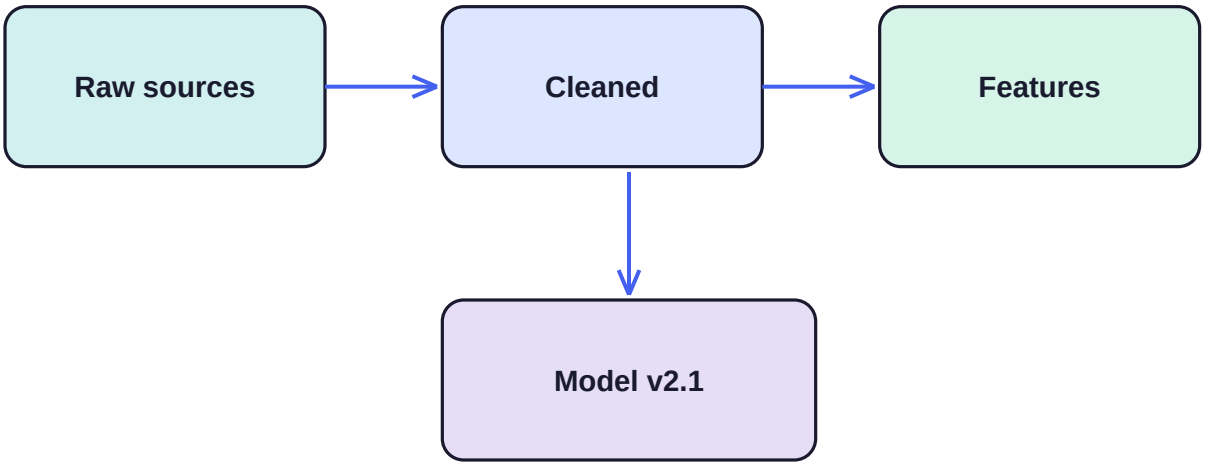
hyperparams + seeds

Miss one ingredient and the result is folklore, not science.

STEP 5/5

Data Lineage

Trace every model back to its raw sources.



Lineage answers: which data trained this model? A bad upstream fix becomes a targeted retrain, not a panic.

Regulators increasingly demand lineage for high-stakes models. Build it before you're asked.

STEP ✓I✓

Key Takeaways

Data Versioning — Remember

- Version data alongside code, or results aren't reproducible.
- DVC: git pointers + content-addressed remote storage.
- LakeFS: branch/merge workflows for data lakes, zero-copy branches.
- Reproducibility = pinned code + data + environment + config.
- Lineage traces models to sources — essential for debugging and compliance.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev